

---

# Task W05: Doubly-Linked Lists

There and back again.

---

## A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

---

In class, we started converting our reference-based list from a singly linked list into a non-circular doubly linked list. The `Node` class is already fully updated, it has both a `prev` and a `next` field. The `add` method is also already correct: it properly sets `prev` and `next` on every insertion, at the head, in the middle, and at the end. You do not need to fix `add`, though it would not hurt to trace through it yourself and confirm that for your own understanding.

The `remove` method, however, was never converted. It still behaves like the old singly linked version, and it will produce a list with broken `prev` pointers the moment you call it. Finding and fixing that is the first part of this lab.

**A note on language.** This lab works the same way in Java and in C/C++. The concepts, and the bugs, are identical either way; only the exact syntax differs. A few reminders for C/C++: `Node` is already a member of your templated class, so a new field is declared as `Node* tail;`, not `Node<T>* tail;`. There is no `public` keyword on individual methods, `nullptr` takes the place of `null`, and because C/C++ has no garbage collector, `remove` has to `delete` the node it removes, which it already does today, that part does not change.

**A note on working with pointers and references.** Every method in this lab moves references, or in C/C++, literal pointers, between nodes. A mistake here does not always announce itself the way a typo or simple logic error does. Sometimes it produces a wrong answer that looks almost right; sometimes nothing looks wrong at all until much later, somewhere that seems unconnected to whatever you actually changed. This is true in both languages, but a C/C++ program is more exposed because the programmer is expected to manage most of the memory, so the same category of mistake can also produce a crash, and not necessarily where you would expect one. If something in your program is behaving strangely, in a way that does not seem to follow logically from whatever you just changed, check your pointer or reference bookkeeping before you look anywhere else. It is one of the most common sources of exactly that kind of confusing failure, in any language.

## 1 Part 1: Finish the Conversion

Two things need to happen here: fixing `remove`, and adding a `tail` reference to the class. Do both; they interact with each other.

---

## Fix remove

Right now, **remove** only touches **next** pointers. Trace through what it does by hand, in both of its cases (removing the first node, and removing any other node), and you will find that it leaves a node's **prev** field pointing at a node that was just removed from the list. Fix both cases so that every node's **prev** field is correct after a removal, the same way **add** already keeps **prev** correct after an insertion.

## Add a tail reference

Add a new field to the class that always points at the last node. A **tail** reference only helps if it is actually kept correct everywhere the list can change, so update all of the following:

- The constructor: a new, empty list has no tail.
- **add**: when the new node ends up with nothing after it, it is the new tail. This can happen when the list was empty before the call, or when inserting after the last node.
- **remove**: when the node being removed was the last node in the list, the node before it becomes the new tail. If removing the node empties the list entirely, the tail becomes null too, the same way the head does.
- **removeAll**: an empty list has no tail, the same way it has no head.

A good way to check your work: after any sequence of adds and removes, the node your **tail** field points to should always be reachable by starting at **head** and following **next** until you run out of nodes, and it should be the same node either way.

## 2 Part 2: New Methods

Add the following methods to the class.

- **displayBackward**  
Prints every item in the list starting from the last one and ending at the first, the same output as **display** but reversed.
- **addFirst**  
Adds an item to the front of the list.
- **addLast**  
Adds an item to the end of the list. You may not implement **addLast** by calling **add**.
- **removeFirst**  
Removes the first item in the list.
- **removeLast**  
Removes the last item in the list.

## 3 Part 3: Put Your List to Work

Add these two methods as well. Do not copy the items into another container (an **ArrayList** in Java, a **vector** in C++, or anything similar) to solve these for you, the point is to use the structure you just finished building.

- 
- **isPalindrome**  
Returns true if the list reads the same forward and backward, and false otherwise. An empty list and a one-item list both count as palindromes.
  - **reverse**  
Reverses the list in place, so that the last item becomes the first and the first becomes the last. When you are done, **head** and **tail** should have swapped roles, and every node's **prev** and **next** should be correct for the new order.

A testing tip: the **Droid** class gives you a way to compare two droids for equality by name (Java's **equals**, or an overloaded **operator==** in C/C++), so you can build a palindrome on purpose to test with. A list containing R2-D2, BB-8, R2-D2 in that order is a genuine palindrome as far as **isPalindrome** is concerned, even though two separate droid objects are involved, one representing each end. Building a few small test cases like this, including an empty list and a single-item list, will catch more bugs than testing only with a roster built the way your driver already builds one.

## 4 Project Setup and Submission

You have been given a folder named **labw05** with the project already set up: for Java, this is an Eclipse project with the package structure already in place; for C/C++, this is the source layout together with the **Makefile**. Do your work inside this folder as given. Do not rename it or rearrange its structure, the layout is part of what gets submitted.

The driver (**DroidDriver.java** for Java, the equivalent **main** for C/C++) already reads a list of droids from a file and loads them into your list before your own code runs. You are expected to add to it: write your own testing code below the point marked in the file, using the loaded droids and any small test cases of your own, to check that your implementation of Parts 1 through 3 actually works.

If you are working in Java, submit the entire **labw05** Eclipse project, not just the individual **.java** files.

If you are working in C/C++, submit the entire **labw05** folder, including the **Makefile** and the rest of the project layout, not just **LinkedList.h** on its own.